SIGPN-T v2.0 — Dossier de livraison de la modernisation

Police Nationale du Tchad · branche `feature/sigpn-modernisation` · cahier des charges v2.0 du 7 novembre 2026 · novembre 2026

1. Résumé
2. Architecture de la modernisation
3. Migrations p. 2
4. Tests et résultats p. 2
5. Manuel d'utilisation (par fonction) p. 3
6. Plan de déploiement p. 4
7. Risques, limites et restes à faire p. 4
8. Ce que contient la démonstration pour la Direction p. 5

Annexe A — Fichiers ajoutés ou modifiés

1. Résumé

Le système existant n'a pas été recréé : il a été **modernisé par couches**, sur la même base de données, avec les mêmes comptes, les mêmes API, les mêmes modules et les mêmes règles métier. Aucune migration n'est destructive ; toutes les données de démonstration sont fictives.

Phase	Contenu	Livraison
1	Audit de l'existant (modules, base, API, droits, risques)	`docs/AUDIT_TPMS_2026-09-12.md`, `docs/phases/PHASE_00_REPORT.md`, `PHASE_01_REPORT.md`
2	Charte graphique marine et or, thème sombre, navigation	commit `13d5123`
3	Tableau de bord opérationnel sur données réelles (périodes, indicateurs, graphiques)	commit `34e6042`
4	Flux des personnes recherchées : brouillon → mandat → vérification → approbation, règle des quatre yeux	commit `35843db`
5	Réseau national d'alertes : file durable, accusés, supervision, relances, escalades	commit `d2c27b1`, `docs/ALERTES_NATIONALES.md`
6	Contrôle biométrique **simulé** : alerte rouge, décision humaine, traçabilité	commit `4bd208b`, `docs/CONTROLE_BIOMETRIQUE.md`
7	Portail public : plaintes, suivi, annuaire, avis publiés, actualités	commit `0cf1c53`, `docs/PORTAIL_PUBLIC.md`
8	Tests de droits, de sécurité, de reprise, de performance, d'accessibilité	commit `3e4c02a`, `docs/TESTS_PHASE8.md`
9	Démonstration pour la Direction générale, captures, ce dossier	ce commit, `docs/DEMO_DG.md`

Au total, par rapport à la branche principale : **8 commits de modernisation, environ 125 fichiers, 5 migrations additives** (liste en annexe).

2. Architecture de la modernisation

2.1 Socle (inchangé)

Next.js 16 (App Router) et React 19 ; Prisma 7 sur PostgreSQL 18 ; Tailwind 4 ; validation zod 4 ; interface française et arabe (de droite à gauche) ; contrôle d'accès par rôles et permissions avec périmètre territorial ; journal d'audit ; second facteur (TOTP) pour les rôles privilégiés. Déploiement conteneurisé (`deploy/`) : base, application, reverse proxy nginx, sauvegarde.

2.2 Ce qui s'ajoute

Parcours d'une requête :

1. **Agent** (navigateur avec session et jeton CSRF) : pages internes `/wanted`, `/alerts`, `/alerts/supervision`, `/biometric/control`, `/public-admin`. **Citoyen** (sans compte, sans cookie) : pages publiques et `/api/v1/public/*`, avec liste blanche de champs et plafonds de débit par adresse.
2. Toute route REST passe par l'enveloppe `withApi({ permission })` : **401** sans identifiant, **403** sans la permission, plafonds `api` (240 par minute) et `apiWrite` (60 par minute) par acteur ou par adresse.
3. Les **services** (`src/server/wanted`, `alerts`, `biometric`, `public`) écrivent dans PostgreSQL par transactions Prisma ; la base impose elle-même les contraintes `CHECK`, les déclencheurs d'immuabilité et le verrou consultatif de la maintenance.

TECHNO-TCHAD

4. Un **planificateur interne** exécute une passe de maintenance des alertes toutes les minutes (ou `npm run alerts:maintain` depuis cron, avec `ALERT_SCHEDULER=off`).

Brique	Rôle	Fichiers principaux
Flux des avis de recherche	Brouillon → mandat actif → vérification (≠ émetteur) → approbation (≠ émetteur) → actif ; suspension, retrait, expiration	`src/server/wanted/`, migrations `…wanted_workflow*`
Réseau d'alertes	Un **événement national immuable** par changement d'état, **une livraison par unité** ; file durable en base, reprise par numéro de séquence	`src/server/alerts/`, `src/app/api/v1/alerts/`, `src/lib/alert-escalation.ts`
Présence des unités	Battement de cœur, fenêtre « connectée » de 2 minutes, coupure simulée pour la démonstration	table `StationPresence`
Maintenance des alertes	Relances, nouvelles tentatives, escalades ; **exclusive** grâce à un verrou consultatif PostgreSQL (aucun doublon avec plusieurs instances) ; plafonds par passe	`src/server/alerts/maintenance.ts`
Contrôle biométrique simulé	Quatre états, seuils de démonstration, décision humaine, signalement d'erreur, lien avec l'alerte du réseau	`src/server/biometric/`, `src/lib/biometric-control.ts`
Portail public	Dépôts, suivi par référence et code (empreinte seule en base), annuaire vérifié, avis publiés après validation expresse, actualités	`src/app/(public)/`, `src/server/public/`, `src/app/api/v1/public*/`
Sécurité transversale	En-têtes (CSP, `X-Frame-Options`, `nosniff`), rejet des octets NUL, plafonds de débit, absence de HTML brut	`src/proxy.ts`, `src/server/api.ts`

2.3 Règles qui tiennent par construction (pas seulement par l'interface)

- L'avis publié sur une alerte est **immuable** (déclencheur en base) : une correction est un nouvel événement.
- Un avis qui n'est pas **actif** ne peut pas être public ; une unité ne peut pas figurer à l'annuaire public sans date de vérification (contraintes `CHECK`).
- L'**émetteur** ne peut ni vérifier ni approuver son propre avis ; un mandat de recherche exige un mandat **actif** lié.
- Une correspondance biométrique est toujours présentée comme une **indication** nécessitant une vérification humaine ; l'écran affiche en permanence qu'il s'agit d'une simulation.
- Le contenu transmis dans l'alerte est minimal ; les pièces restent dans leur périmètre d'accès.

3. Migrations

Toutes sont **additives** (aucun `DROP TABLE`, `DROP COLUMN`, `TRUNCATE` ni `DELETE`) ; la procédure de retour arrière figure en tête de chaque fichier et ne doit être exécutée qu'après sauvegarde.

Migration	Contenu
`20261108100000_wanted_workflow`	États, vérification, approbation, mandat lié des avis de recherche
`20261108100100_wanted_workflow_columns`	Colonnes complémentaires du flux
`20261108120000_national_alerts`	`NationalAlert`, `AlertDelivery`, `StationPresence`, niveau de diffusion
`20261108140000_biometric_control`	Contrôles biométriques simulés et leurs décisions
`20261108160000_public_portal`	Dépôts publics, actualités, champs d'annuaire, publication expresse

Vérification effectuée : rejeu de l'ensemble des migrations sur une base vide, puis comparaison du schéma avec la base de développement (1 560 colonnes, 445 index, 1 458 contraintes, 45 déclencheurs, 112 énumérations) : **identique**. Le détail est dans `docs/TESTS_PHASE8.md`.

4. Tests et résultats

Domaine	Moyen	Résultat
Typage	`tsc --noEmit`	sans erreur
Unitaires	`npm test`	130 sur 130
Intégration et sécurité (serveur et base réels)	`npm run test:integration` et `npm run test:security`	215 sur 215 (dont 24 ajoutés en phase 8)
Droits	matrice générée depuis le code de **toutes** les routes REST : 401 sans identifiant, 403 sans permission, jamais 5xx	réussi
Sécurité du site public	injections, affectation de masse, corps mal formés, identifiants hostiles, en-têtes	réussi ; un défaut corrigé (octet NUL → 500, désormais 422)

TECHNOTICIAD

Reprise du réseau d'alertes	longue panne, 25 synchronisations et 10 accusés simultanés, clé révoquée	réussi ; un défaut corrigé (passes de maintenance simultanées : 42 escalades pour 14 livraisons, désormais exclusives)
Performance (300 unités simulées, 20 avis, 6 000 livraisons, build de production)	`npm run bench:p8`	publication 138 ms (95e centile), supervision 27 ms, battement 31 ms, pages publiques 270 ms au plus ; tous les budgets respectés
Démonstration	`npm run demo:run` : cinq scénarios par l'interface réelle, 30 captures	réussi sur une base neuve

Relance complète en phase 9 (après ajout des comptes d'Abéché, des scripts de démonstration et de la correction de la graine) : `tsc` sans erreur, **130 sur 130** et **215 sur 215**. Deux fragilités **des tests** (non du produit) ont été corrigées au passage : (1) trois suites échouaient en 429 quand elles suivaient `hr-security`, qui épuise exprès le quota d'écriture de 60 par minute du compte administrateur : leurs aides d'appel attendent désormais la fin de la fenêtre ; (2) le test des passes de maintenance simultanées comparait le total d'escalades de tout le réseau aux livraisons d'un seul avis, ce qui échouait dès que des avis d'exécutions précédentes subsistaient : la borne est maintenant le nombre de livraisons éligibles avant les passes.

Le rapport détaillé est `docs/TESTS_PHASE8.md` (et son PDF).

5. Manuel d'utilisation (par fonction)

Chaque fonction dispose du menu correspondant ; les boutons absents indiquent un droit manquant.

5.1 Enquêteur — créer un avis de recherche

1. *Personnes recherchées* → *Nouvel avis* : choisir la personne, saisir l'infraction, la nature, le **périmètre de diffusion** (nationale, régionale, locale), la priorité, l'autorité émettrice, la base juridique et le motif (interne, jamais publié).
2. Enregistrer : l'avis est en **brouillon**. S'il s'agit d'un mandat de recherche, remplir *Demande de mandat*.
3. Un autre agent vérifiera ; l'enquêteur ne peut ni vérifier ni approuver son propre avis.

5.2 Vérificateur (commissaire de poste) — vérifier

Ouvrir l'avis en brouillon, contrôler les pièces et la base juridique, ajouter des observations, **Valider la vérification**. L'avis passe en attente d'approbation.

5.3 Autorité — activer le mandat et approuver

Sur la fiche : activer le mandat lié, puis changer le statut vers **Actif**. L'approbation diffuse l'alerte à toutes les unités du périmètre choisi.

5.4 Agent d'unité — recevoir une alerte

Le bandeau rouge « alerte nationale à traiter » apparaît sur toutes les pages dans les 20 secondes. *Alertes nationales* : **Marquer comme lu**, puis **Acquitter**. Un signal sonore bref peut être activé par poste. Une alerte n'établit jamais la culpabilité : vérification humaine avant toute action.

5.5 Agent habilité au contrôle biométrique

Contrôle biométrique : choisir la modalité (empreinte ou visage), saisir le motif du contrôle, lancer. En cas de correspondance possible : **Voir le dossier autorisé**, **Demander vérification**, **Notifier le supérieur** ou **Signaler une erreur**. L'écran est une simulation tant qu'aucun système réel n'est branché.

5.6 Superviseur (commandement) — supervision nationale

Supervision des alertes : filtres par période, province, unité, statut et type. Le détail d'une alerte liste chaque unité : reçue, lue, acquittée, tentatives, escalade. **Relancer les unités** force une passe de relance. Les boutons *Couper l'unité* et le formulaire d'exercice (rôles `alerts:simulate`) servent à la formation et à la démonstration ; les exercices sont marqués comme tels.

5.7 Gestionnaire du site public

Site public : trois onglets. **Dépôts** : ouvrir, changer l'état, rédiger la réponse publique, ou convertir en plainte interne. **Actualités** : rédiger puis publier (une seconde personne publie, sauf rôle disposant des deux droits). **Annuaire** : adresse, horaires, téléphone ; la vérification est obligatoire avant l'affichage public. **Publication d'un avis** : sur la fiche d'un avis actif et vérifié, rédiger le texte public, cocher la validation expresse, publier ; le retrait est automatique dès que l'avis cesse d'être actif.

5.8 Citoyen (sans compte)

Portail public : *Déposer une plainte*, *Signalement*, *Suivre ma demande* (référence et code, notés à l'enregistrement : le code ne s'affiche qu'une fois), *Commissariats*, *Avis de recherche*, *Actualités*, *Contact*. Ne jamais saisir de données d'un tiers sans raison : le formulaire rappelle qu'il ne remplace pas l'intervention d'urgence.

TECHNOTCHAD

6. Plan de déploiement

6.1 Préparer

1. **Serveur** : Linux avec Docker et Docker Compose, disque dimensionné pour la base, les pièces et les sauvegardes ; heure synchronisée (NTP) — les codes à second facteur et les délais d'escalade en dépendent.
2. **Secrets** : générer `AUTH_SECRET` et `DATA_ENCRYPTION_KEY` (32 octets aléatoires chacun), mot de passe de base. Les conserver hors du dépôt ; **perdre `DATA_ENCRYPTION_KEY` rend les secrets de second facteur illisibles**.
3. **Réseau** : reverse proxy nginx (`deploy/nginx`) avec certificat TLS ; `TRUST_PROXY=1` **uniquement** derrière ce proxy de confiance, sinon tout le trafic partage une même adresse et les plafonds de débit du site public deviennent globaux.
4. **Variables** : fichier `.env` d'après `.env.example` (`APP_URL`, `ALERT_SCHEDULER`, `ALERT_SCHEDULER_INTERVAL_MINUTES`, `PUBLIC_SUBMIT_LIMIT`, `PUBLIC_TRACK_LIMIT`, durée de session, verrouillage).

6.2 Mettre en service (hors heures de pointe)

1. **Sauvegarder la base actuelle** et **vérifier qu'elle se restaure** sur une copie (voir 6.3) : c'est la condition de la bascule.
2. `docker compose -f deploy/docker-compose.yml up -d --build` : le conteneur applique `prisma migrate deploy` au démarrage (les cinq migrations additives), puis lance l'application.
3. Premier administrateur : variables `BOOTSTRAP_ADMIN_*` au premier démarrage uniquement, puis les retirer ; changement de mot de passe et enrôlement du second facteur sont forcés.
4. **Ne pas** lancer la graine de démonstration en production : elle crée des comptes fictifs.
5. Contrôles de recette : `/api/health`, connexion et second facteur, création d'un avis de test dans une unité pilote, réception et acquittement, publication sur le site public d'un avis de test puis retrait.
6. Passer le site public derrière son nom de domaine ; l'indexation par les moteurs de recherche est **désactivée** tant qu'elle n'a pas été décidée.

6.3 Sauvegarde et restauration

- Quotidienne : le service `backup` du fichier Compose exécute `deploy/backup/backup.sh` (`pg_dump` au format personnalisé, empreinte SHA-256, rétention 30 jours).
- Depuis un poste : `npm run db:backup` et `npm run db:restore -- <fichier>` appellent ces mêmes scripts ; ils exigent `pg_dump`/`pg_restore` et les variables `PGHOST`, `PGUSER`, `PGPASSWORD`, `PGDATABASE`. **La restauration supprime et recrée la base : arrêter l'application d'abord.**
- Les pièces (volume `uploads`) se sauvegardent séparément (copie du volume).
- **Un cycle complet sauvegarde → restauration n'a pas été exécuté sur le poste de développement** (outils client PostgreSQL absents). Il doit être réalisé et chronométré sur le serveur cible avant la bascule.

6.4 Retour arrière

Réinstaller l'image précédente et restaurer la sauvegarde d'avant mise en service. Les cinq migrations étant additives, l'ancienne version fonctionne aussi sur la nouvelle base ; les procédures de retrait des tables ajoutées sont en tête de chaque migration.

6.5 Conduite du changement

Formation par fonction (section 5), période de double saisie de deux semaines sur une province pilote, désignation d'un responsable du **site public** (qui valide les publications) et d'un responsable de la **supervision des alertes**, définition des délais de relance et d'escalade propres à la Direction.

7. Risques, limites et restes à faire

Sujet	État	Action
Biométrie	**Simulée.** Seuils de démonstration	Base légale, système homologué, tests de précision, analyse d'impact avant tout branchement
Délai des alertes	Environ 20 secondes (synchronisation périodique), pas de notification instantanée ni sur téléphone	Décider d'un canal push (WebSocket ou mobile) si le délai est insuffisant
Coupure réelle de liaison	Simulée dans l'application ; reprise démontrée par construction et par les tests, **pas** par un arrêt brutal ni une vraie coupure (câble, VSAT)	Essai terrain sur une unité pilote
Charge	300 unités simulées sur **un seul poste**, 18 comptes de sondage ; délais réseau non mesurés	Essai sur l'infrastructure réelle
Sauvegarde	Scripts présents, `npm run db:backup`/`db:restore` désormais rattachés à eux ; **cycle non exécuté ici**	Répétition chronométrée sur le serveur cible

Photographie	Non diffusée dans les alertes ni sur le site public	Décision de la Direction et circuit d'autorisation
Texte figé des alertes	Le bulletin est immuable : sa mention « vérification humaine obligatoire » reste en français dans l'interface arabe	Prévoir une version arabe du texte à la publication
Site public	Pas d'e-mail ni de SMS de confirmation, pas de CAPTCHA, pas de pièce jointe ; l'indexation des moteurs de recherche est désactivée	À décider selon l'usage réel et les abus constatés
Accessibilité	Audit scripté à 375 px (FR, AR) : aucun débordement ni champ sans libellé ; cibles tactiles agrandies. **Pas** d'essai sur appareil réel, de lecteur d'écran ni de mesure de contraste automatisée ; quelques liens de tableaux de moins de 24 px	Audit d'accessibilité complet
Sécurité	Tests internes seulement	Audit externe et test d'intrusion avant mise en service
Données de démonstration	Fictives ; l'annuaire et les actualités de la démonstration ne contiennent aucun numéro officiel	À remplacer par des contenus vérifiés avec chaque unité
Graine de démonstration	Défaut corrigé en phase 9 : la graine des avis de recherche violait la règle « l'approbateur n'est pas l'émetteur » (contrainte de base ajoutée en phase 4) et s'arrêtait à la première exécution sur base vide	Aucune ; vérifié sur base vide

8. Ce que contient la démonstration pour la Direction

`docs/DEMO_DG.md` (et son PDF) : préparation, comptes, cinq scénarios illustrés de 30 captures, messages clés, limites à ne pas taire, questions probables. Les scripts `scripts/demo/*` reconstruisent la base de démonstration (`npm run demo:reset`), activent le second facteur des comptes de direction (`npm run demo:mfa`) et rejouent les cinq scénarios par l'interface (`npm run demo:run`) ; ils refusent toute base dont le nom ne se termine pas par `_demo`.

Nouveaux comptes fictifs ajoutés à la graine pour la démonstration d'Abéché : `chief.abe` (commissaire de poste) et `desk.abe` (agent d'accueil), rattachés au Commissariat Central d'Abéché.

Annexe A — Fichiers ajoutés ou modifiés

Comparaison avec la branche principale : **168 fichiers** (129 ajoutés, 39 modifiés). `A` = ajouté, `M` = modifié.

Base de données (migrations et schéma) — 6

	Fichier
A	`prisma/migrations/20261108100000_wanted_workflow/migration.sql`
A	`prisma/migrations/20261108100100_wanted_workflow_columns/migration.sql`
A	`prisma/migrations/20261108120000_national_alerts/migration.sql`
A	`prisma/migrations/20261108140000_biometric_control/migration.sql`
A	`prisma/migrations/20261108160000_public_portal/migration.sql`
M	`prisma/schema.prisma`

Services serveur (`src/server`) — 22

	Fichier
A	`src/server/alerts/actions.ts`
A	`src/server/alerts/maintenance.ts`
A	`src/server/alerts/publish.ts`
A	`src/server/alerts/scheduler.ts`
A	`src/server/alerts/service.ts`
M	`src/server/api.ts`
M	`src/server/auth/rate-limit.ts`
A	`src/server/biometric/control.ts`
A	`src/server/dashboard/stats.ts`
M	`src/server/demo/accounts.ts`
M	`src/server/demo/seed-wanted.ts`
A	`src/server/public/actions.ts`
A	`src/server/public/admin.ts`

TECHNOTCHAD

A	src/server/public/service.ts
M	src/server/rbac/permissions.ts
M	src/server/rbac/roles.ts
M	src/server/rbac/sync.ts
M	src/server/sequences.ts
M	src/server/wanted/actions.ts
M	src/server/wanted/queries.ts
M	src/server/wanted/schemas.ts
M	src/server/wanted/service.ts

Routes REST (`src/app/api`) — 25

	Fichier
A	src/app/api/v1/alerts/[id]/ack/route.ts
A	src/app/api/v1/alerts/[id]/read/route.ts
A	src/app/api/v1/alerts/[id]/route.ts
A	src/app/api/v1/alerts/heartbeat/route.ts
A	src/app/api/v1/alerts/route.ts
A	src/app/api/v1/alerts/simulate/route.ts
A	src/app/api/v1/alerts/supervision/[id]/remind/route.ts
A	src/app/api/v1/alerts/supervision/[id]/route.ts
A	src/app/api/v1/alerts/supervision/route.ts
A	src/app/api/v1/alerts/units/[id]/presence/route.ts
A	src/app/api/v1/biometric/control/[id]/route.ts
A	src/app/api/v1/biometric/control/route.ts
A	src/app/api/v1/intake/[id]/route.ts
A	src/app/api/v1/intake/route.ts
A	src/app/api/v1/public-content/news/[id]/route.ts
A	src/app/api/v1/public-content/news/route.ts
A	src/app/api/v1/public-content/stations/[id]/route.ts
A	src/app/api/v1/public-content/stations/route.ts
A	src/app/api/v1/public/news/route.ts
A	src/app/api/v1/public/stations/route.ts
A	src/app/api/v1/public/submissions/route.ts
A	src/app/api/v1/public/track/route.ts
A	src/app/api/v1/public/wanted/route.ts
M	src/app/api/v1/wanted/[id]/route.ts
M	src/app/api/v1/wanted/route.ts

Pages internes (`src/app/(app)`) — 14

	Fichier
A	src/app/(app)/alerts/[id]/page.tsx
A	src/app/(app)/alerts/alerts-forms.tsx
A	src/app/(app)/alerts/page.tsx
A	src/app/(app)/alerts/supervision/[id]/page.tsx
A	src/app/(app)/alerts/supervision/page.tsx
A	src/app/(app)/biometric/control/control-console.tsx
A	src/app/(app)/biometric/control/page.tsx
M	src/app/(app)/dashboard/page.tsx

TECHNOTOCHAD

M	src/app/(app)/layout.tsx
A	src/app/(app)/public-admin/page.tsx
A	src/app/(app)/public-admin/public-admin-forms.tsx
A	src/app/(app)/public-admin/submissions/[id]/page.tsx
M	src/app/(app)/wanted/[id]/page.tsx
M	src/app/(app)/wanted/wanted-forms.tsx

Portail public (`src/app/(public)`) — 11

	Fichier
A	src/app/(public)/actualites/[id]/page.tsx
A	src/app/(public)/actualites/page.tsx
A	src/app/(public)/avis-de-recherche/page.tsx
A	src/app/(public)/commissariats/page.tsx
A	src/app/(public)/contact/page.tsx
A	src/app/(public)/layout.tsx
A	src/app/(public)/page.tsx
A	src/app/(public)/plaintes/page.tsx
A	src/app/(public)/public-form.tsx
A	src/app/(public)/signalement/page.tsx
A	src/app/(public)/suivi/page.tsx

Composants, thème et navigation — 9

	Fichier
M	src/app/globals.css
A	src/components/app-shell/alert-watcher.tsx
M	src/components/app-shell/sidebar.tsx
M	src/components/charts.tsx
M	src/components/language-switcher.tsx
M	src/instrumentation.ts
M	src/lib/labels.ts
M	src/lib/navigation.ts
M	src/proxy.ts

Règles métier partagées et traductions (`src/lib`) — 17

	Fichier
A	src/lib/alert-escalation.ts
A	src/lib/biometric-control.ts
A	src/lib/dash-period.ts
A	src/lib/i18n/alerts.ar.ts
A	src/lib/i18n/alerts.fr.ts
M	src/lib/i18n/ar.ts
A	src/lib/i18n/biocontrol.ar.ts
A	src/lib/i18n/biocontrol.fr.ts
M	src/lib/i18n/catalog.ts
A	src/lib/i18n/dash.ar.ts
A	src/lib/i18n/dash.fr.ts
M	src/lib/i18n/errors.ts
M	src/lib/i18n/fr.ts

TECHNO TCHAD

A	`src/lib/i18n/public.ar.ts`
A	`src/lib/i18n/public.fr.ts`
A	`src/lib/i18n/wanted.ar.ts`
A	`src/lib/i18n/wanted.fr.ts`

Tests — 12

	Fichier
A	`tests/integration/alerts-resilience.test.ts`
A	`tests/integration/alerts.test.ts`
A	`tests/integration/biocontrol.test.ts`
A	`tests/integration/public.test.ts`
M	`tests/integration/users-api.test.ts`
M	`tests/integration/wanted.test.ts`
A	`tests/security/public-attacks.test.ts`
A	`tests/security/route-matrix.test.ts`
A	`tests/unit/alert-escalation.test.ts`
A	`tests/unit/biometric-control.test.ts`
A	`tests/unit/dash-period.test.ts`
A	`tests/unit/public-boundary.test.ts`

Outils et scripts — 7

	Fichier
A	`scripts/alerts-maintain.ts`
A	`scripts/dash-check.ts`
A	`scripts/demo/lib.ts`
A	`scripts/demo/mfa.ts`
A	`scripts/demo/reset.ts`
A	`scripts/demo/run.ts`
A	`scripts/p8-bench.ts`

Documentation — 9

	Fichier
A	`docs/ALERTES_NATIONALES.md`
A	`docs/CONTROLE_BIOMETRIQUE.md`
A	`docs/DEMO_DG.md`
A	`docs/DEMO_DG.pdf`
A	`docs/LIVRAISON_FINALE.md`
A	`docs/LIVRAISON_FINALE.pdf`
A	`docs/PORTAIL_PUBLIC.md`
A	`docs/TESTS_PHASE8.md`
A	`docs/TESTS_PHASE8.pdf`

Captures de la démonstration — 30

30 images PNG dans `docs/demo/screens/` (scénarios 1 à 5).

Configuration et divers — 6

	Fichier
M	`.env.example`
M	`.gitignore`

M	`deploy/docker-compose.yml`
M	`next.config.ts`
M	`package-lock.json`
M	`package.json`

TECHNOTCHAD